# Deploying the Semsar Backend to a Hostinger VPS (CyberPanel)

This guide walks through putting the Semsar (Aveno Marketplace) Laravel 11 API — including the admin panel — on a Hostinger VPS running **CyberPanel** (OpenLiteSpeed). It assumes root SSH access to the VPS and a domain or subdomain you can point at it (e.g. `api.yourdomain.com`).

Repository: `https://github.com/Nasr4IT/-aveno-simsar-api.git`

---

## 1. What this app needs

| Requirement | Value |
|---|---|
| PHP | 8.2 or newer |
| PHP extensions | openssl, pdo, mbstring, tokenizer, xml, ctype, json, bcmath, fileinfo, curl, zip, **gd** |
| Database | MySQL 5.7+/8 (or PostgreSQL — this guide uses MySQL since that's what CyberPanel provisions out of the box) |
| Web server | OpenLiteSpeed, document root = the app's `public/` folder (**not** the project root) |
| Node/npm | Not needed — this is an API + server-rendered admin panel, there's no frontend build step |
| Background process | One `php artisan queue:work` process must run continuously (push notifications, approval emails, etc. are queued) |

---

## 2. Point the domain at the VPS

In your domain's DNS settings, add an **A record** pointing the subdomain you'll use (e.g. `api.yourdomain.com`) at the VPS's public IP address. This can take a few minutes to a few hours to propagate — do it first so it's ready by the time you need SSL later.

---

## 3. Create the website in CyberPanel

1. Log into CyberPanel (`https://<vps-ip>:8090`).

2. **Websites** → **Create Website**.
3. Pick the domain/subdomain from step 2.
4. PHP version: choose **8.2** or newer.
5. Leave SSL off for now (you'll issue a free one after the domain resolves).
6. Create the website.

This gives you a system user and a home directory at `/home/<domain>/`.

---

## 4. SSH in and get the code

```
ssh root@<vps-ip>

# Composer should already be on a CyberPanel box; if not:
curl -sS https://getcomposer.org/installer | php
mv composer.phar /usr/local/bin/composer

# Clone the app OUTSIDE public_html — public_html will only ever point at
# the app's public/ subfolder (step 7), never the project root. This keeps
# .env, app/, vendor/, etc. unreachable from the web.
cd /home/<domain>
git clone https://github.com/Nasr4IT/-aveno-simsar-api.git backend
cd backend

composer install --no-dev --optimize-autoloader
```

---

## 5. Create the database

In CyberPanel: **Databases** → **Create Database**. Note the database name, username, and password it gives you — you'll need them in the next step.

---

## 6. Configure `.env`

```
cp .env.example .env
nano .env
```

Set at minimum:

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://api.yourdomain.com

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=<the database name from step 5>
DB_USERNAME=<the database user from step 5>
DB_PASSWORD=<the database password from step 5>

FILESYSTEM_DISK=public
QUEUE_CONNECTION=database
```

Everything else in `.env.example` (Sham Cash, Firebase push notifications, Sentry, S3/R2 storage) is **optional and safe to leave blank** — the app was built to no-op gracefully without those credentials. Fill them in later if/when you have real accounts for them; nothing breaks in the meantime.

Then generate the app key (do this on the server, never commit a real key):

```
php artisan key:generate --force
```

---

## 7. Point the website's document root at `public/`

Laravel's entire security model depends on the web server only ever serving the `public/` folder, not the project root. In CyberPanel:

1. **Websites** → **List Websites** → **Manage** (for your domain).
2. **vHost Conf** tab.
3. Find the line starting with `docRoot` and change it to:

   ```
   docRoot                   /home/<domain>/backend/public
   ```

4. Save, then restart OpenLiteSpeed (CyberPanel will prompt you, or run `systemctl restart lsws` over SSH).

---

## 8. File permissions

The web server user needs write access to `storage/` and `bootstrap/cache/` (logs, cached views/config, uploaded images):

```
cd /home/<domain>/backend
chown -R <domain-system-user>:<domain-system-user> storage bootstrap/cache
chmod -R 775 storage bootstrap/cache
```

(The system user is whatever CyberPanel created for this website in step 3 — check **Websites** → **List Websites** if you're not sure of the exact name.)

---

## 9. Migrate, seed, and link storage

```
php artisan migrate --force
php artisan db:seed --force
php artisan storage:link
```

`db:seed` creates the category tree (with its dynamic attributes) and **one default admin account** so you can log into the admin panel immediately:

- Phone: `0999999999`
- Password: `change-me-now`

**Change this password immediately** once you've confirmed the panel works — it's a published default, not a secret.

`storage:link` makes uploaded ad photos/banners reachable over the web ( `public/storage` → `storage/app/public` ). Skip this only if you later switch `FILESYSTEM_DISK` to `s3` for Cloudflare R2/AWS S3-backed storage instead.

---

## 10. Check PHP extensions

Laravel will fail loudly if something's missing, but it's worth checking `gd` specifically (used for image resizing) before you go further:

```
php -m | grep -i gd
```

If it's missing, install it for whichever `lsphp` version you picked in step 3 (e.g. `lsphp82-gd` ) via CyberPanel's **PHP** → **Select PHP** → **Extensions** screen, or:

```
apt install lsphp82-gd   # adjust the version number to match your site's PHP
systemctl restart lsws
```

---

## 11. Issue a free SSL certificate

Once the DNS A record from step 2 has propagated:

1. **SSL** → **Manage SSL** for your domain.
2. **Issue SSL** (Let's Encrypt, free, auto-renews).

---

## 12. Verify the rewrite rules (pretty URLs)

Laravel ships its own `public/.htaccess`, and OpenLiteSpeed honors it on most CyberPanel sites out of the box. Test it:

```
curl -I https://api.yourdomain.com/api/categories
```

You should get `HTTP/1.1 200`. If instead you get a 404, OpenLiteSpeed isn't picking up `.htaccess` — add the rewrite rule manually:

1. **Websites** → **Manage** → **Rewrite** tab.
2. Paste:

   ```
   RewriteEngine On
   RewriteCond %{REQUEST_FILENAME} !-f
   RewriteCond %{REQUEST_FILENAME} !-d
   RewriteRule ^ index.php [L]
   ```

3. Save and restart OpenLiteSpeed.

---

## 13. Run the queue worker permanently

Push notifications, approval/rejection emails, and other notifications are **queued**, not sent immediately — something has to keep running `php artisan queue:work` in the background, or they'll just sit unsent in the `jobs` table forever. Set it up as a `systemd` service so it survives reboots and restarts itself if it crashes:

```
nano /etc/systemd/system/semsar-queue.service
```

```
[Unit]
Description=Semsar Laravel Queue Worker
After=network.target mysql.service

[Service]
User=<domain-system-user>
WorkingDirectory=/home/<domain>/backend
ExecStart=/usr/bin/php /home/<domain>/backend/artisan queue:work --tries=3 --sleep=3
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
systemctl daemon-reload
systemctl enable --now semsar-queue
systemctl status semsar-queue   # confirm it's "active (running)"
```

---

## 14. (Optional, future-proofing) Laravel's scheduler

Nothing uses it yet, but it costs nothing to wire up now so it's ready the day something does (e.g. auto-expiring old ads):

```
crontab -e
```

```
* * * * * cd /home/<domain>/backend && php artisan schedule:run >> /dev/null 2>&1
```

---

## 15. Test it

```
curl https://api.yourdomain.com/api/categories
```

Should return a JSON array of categories.

Then open `https://api.yourdomain.com/admin-panel/login` in a browser — you should see the Semsar Admin Panel login screen. Log in with the seeded admin account from step 9, confirm the dashboard loads, **then change that password**.

Give your Flutter developer friend `https://api.yourdomain.com` as the API base URL — `/api/...` for every endpoint in `API_REFERENCE.md` /the Postman collection.

---

## 16. Deploying future updates

Whenever you push new commits to GitHub, redeploy with:

```
cd /home/<domain>/backend
git pull origin main
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:clear
php artisan view:clear
systemctl restart semsar-queue
```

( `config:clear` / `view:clear` avoid serving stale cached config or Blade views after a deploy; `migrate --force` is safe to run every time — Laravel skips migrations that already ran.)

---

## Quick troubleshooting

| Symptom | Likely cause |
|---|---|
| 500 error, blank page | `APP_DEBUG=false` hides the real error — temporarily set it to `true`, reload, then set it back to `false` and check `storage/logs/laravel.log` instead |
| 404 on every `/api/...` route | Rewrite rules not active — see step 12 |
| Ad/banner images 404 | Forgot `php artisan storage:link` (step 9), or wrong file permissions (step 8) |
| Notifications/push never arrive | Queue worker not running — check `systemctl status semsar-queue` |
| "could not find driver" on migrate | The `pdo_mysql` PHP extension isn't enabled for the PHP version this site uses |